

ĐẠI HỌC HUẾ
TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN



BÀI TIỂU LUẬN

ĐỀ TÀI: Điều khiển từ xa qua giao thức MQTT với ESP32

HỌC PHẦN: Phát triển ứng dụng IOT – Nhóm 3

GIẢNG VIÊN HƯỚNG DẪN: ThS.VÕ VIỆT DŨNG

SINH VIÊN THỰC HIỆN:

Huế, Tháng 4 Năm 2026

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ / Giải nghĩa
ADC	Analog-to-Digital Converter – Bộ chuyển đổi tín hiệu tương tự sang số
API	Application Programming Interface – Giao diện lập trình ứng dụng
BLE	Bluetooth Low Energy – Bluetooth năng lượng thấp
BR/EDR	Basic Rate / Enhanced Data Rate – Chế độ Bluetooth cổ điển
C/C++	Ngôn ngữ lập trình C và C++
DAC	Digital-to-Analog Converter – Bộ chuyển đổi tín hiệu số sang tương tự
ESP-IDF	Espressif IoT Development Framework – Framework phát triển IoT của Espressif
ESP32	Vi điều khiển SoC do Espressif Systems phát triển, tích hợp Wi-Fi và Bluetooth
GPIO	General Purpose Input/Output – Chân đầu vào/ra đa năng
HTTP	HyperText Transfer Protocol – Giao thức truyền tải siêu văn bản
I2C	Inter-Integrated Circuit – Giao thức truyền thông nối tiếp hai dây
IDE	Integrated Development Environment – Môi trường phát triển tích hợp
InfluxDB	Cơ sở dữ liệu chuỗi thời gian mã nguồn mở dùng cho IoT và monitoring
IoT	Internet of Things – Internet vạn vật
JSON	JavaScript Object Notation – Định dạng trao đổi dữ liệu nhẹ dựa trên văn bản
LED	Light Emitting Diode – Điốt phát quang
M2M	Machine-to-Machine – Giao tiếp máy với máy
MQTT	Message Queuing Telemetry Transport – Giao thức truyền nhắn tin kiểu publish/subscribe dành cho IoT
MySQL	Hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở
OTA	Over-The-Air – Cập nhật firmware từ xa qua mạng không dây
PlatformIO	Nền tảng phát triển phần mềm nhúng tích hợp trong VSCode
QoS	Quality of Service – Chất lượng dịch vụ (trong truyền thông MQTT)
RAM	Random Access Memory – Bộ nhớ truy cập ngẫu nhiên
SoC	System on Chip – Hệ thống trên một chip
SPI	Serial Peripheral Interface – Giao thức truyền thông nối tiếp đa dây

Từ viết tắt	Tên đầy đủ / Giải nghĩa
SSID	Service Set Identifier – Tên mạng Wi-Fi
STM32	Dòng vi điều khiển ARM Cortex của STMicroelectronics
TCP	Transmission Control Protocol – Giao thức điều khiển truyền tải
UART	Universal Asynchronous Receiver/Transmitter – Giao thức truyền thông nối tiếp không đồng bộ
UI	User Interface – Giao diện người dùng
USB	Universal Serial Bus – Chuẩn kết nối cáp đa năng
VSCode	Visual Studio Code – Trình soạn thảo mã nguồn của Microsoft
Wi-Fi	Wireless Fidelity – Mạng không dây chuẩn IEEE 802.11
Wokwi	Nền tảng mô phỏng phần cứng vi điều khiển trực tuyến

DANH MỤC HÌNH ẢNH

<i>Hình 1: Ảnh thực tế bo mạch ESP32.....</i>	<i>3</i>
<i>Hình 2: Kiến trúc tổng thể hệ thống.....</i>	<i>9</i>
<i>Hình 3: Sơ đồ mô phỏng của hệ thống.....</i>	<i>10</i>
<i>Hình 4: Lưu đồ thuật toán điều khiển.</i>	<i>12</i>
<i>Hình 5: Cấu hình dashboard blynk.....</i>	<i>15</i>
<i>Hình 6: Khi khởi động đèn nhấp nháy.....</i>	<i>18</i>
<i>Hình 7: Kết nối tới wifi, MQTT, Blynk, Telegram.....</i>	<i>19</i>
<i>Hình 8: Điều khiển qua Telegram.....</i>	<i>20</i>
<i>Hình 9: Điều khiển tắt/bật qua blynk và hiển thị trạng thái LED.....</i>	<i>20</i>
<i>Hình 10: Điều khiển qua MQTT.....</i>	<i>21</i>
<i>Hình 11: Đồng bộ trạng thái.....</i>	<i>22</i>

MỤC LỤC

LỜI MỞ ĐẦU	1
I. CƠ SỞ LÝ THUYẾT	2
1.1. Tổng quan về IOT	2
1.2. Vi điều khiển ESP32	2
1.3. Giao thức MQTT	3
1.4. TMQTT Broker (Mosquitto/HiveMQ)	4
1.5. Tổng quan về Telegram	5
1.6. Nền tảng Blynk IoT	5
1.7. So sánh HTTP polling và thư viện native (Blynk lib, UniversalTelegramBot)..	6
1.8. Ứng dụng ESP32 với giao thức MQTT	7
II. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	7
2.1. Yêu cầu chức năng.....	7
2.2. Yêu cầu phi chức năng.....	8
2.3. Kiến trúc tổng thể hệ thống.....	8
2.4. Thiết kế phần cứng.....	9
2.5. Thiết kế phần mềm.....	10
2.6. Lưu đồ thuật toán điều khiển.....	12
2.7. Cơ chế đồng bộ trạng thái đa nền tảng (MQTT – Telegram - Blynk).....	13
2.8. Cơ chế chống lặp lệnh (anti-loop), chống ghi đè trạng thái.....	13
III. Triển khai thực nghiệm	14
3.1. Công cụ và môi trường (Wokwi, PlatformIO, Arduino framework)	14
3.2. Cấu hình Wifi, MQTT, Telegram, Blynk	15
3.3. Cài đặt thư viện và biên dịch.....	16
3.4. Nạp chương trình và vận hành	17
IV. MÔ PHỎNG TRÊN WOKWI VÀ ĐÁNH GIÁ HỆ THỐNG	17
4.1. Giới thiệu Wokwi	17
4.2. Kết quả mô phỏng.....	17
4.3. Đánh giá hệ thống và đề xuất hướng phát triển.....	22
4.3.1. Đánh giá:.....	22
4.3.2 Hướng phát triển:	23
V. KẾT LUẬN	24

LỜI MỞ ĐẦU

Sự phát triển bùng nổ của Internet vạn vật (IoT) đã và đang làm thay đổi sâu sắc cách con người tương tác với thế giới vật lý. Khả năng kết nối, giám sát và điều khiển các thiết bị từ xa thông qua mạng Internet không còn là khái niệm viễn tưởng mà đã trở thành hiện thực trong nhiều lĩnh vực như nhà thông minh, nông nghiệp thông minh, và tự động hóa công nghiệp. Trong bức tranh tổng thể đó, việc lựa chọn một giải pháp truyền thông hiệu quả, tin cậy và phù hợp với các thiết bị có tài nguyên hạn chế là một yếu tố then chốt.

Giao thức MQTT (Message Queuing Telemetry Transport) nổi lên như một tiêu chuẩn hàng đầu cho truyền thông M2M (Machine-to-Machine) và IoT nhờ tính đơn giản, nhẹ và hiệu quả. Kết hợp với ESP32, một vi điều khiển mạnh mẽ, chi phí thấp và được tích hợp sẵn Wi-Fi, chúng ta có một nền tảng lý tưởng để xây dựng các ứng dụng IoT. Để tăng cường tính linh hoạt và trải nghiệm người dùng, các nền tảng như Telegram và Blynk có thể được tích hợp, cung cấp các kênh điều khiển và giám sát trực quan, thuận tiện hơn.

Bài tiểu luận này tập trung vào việc nghiên cứu và triển khai một hệ thống điều khiển thiết bị từ xa (cụ thể là một đèn LED) sử dụng ESP32 làm client. Hệ thống được thiết kế để có thể nhận lệnh điều khiển từ ba nguồn khác nhau: ứng dụng nhắn tin Telegram, nền tảng Blynk IoT và các client MQTT khác, với trạng thái được đồng bộ xuyên suốt. Qua đó, bài viết sẽ làm rõ cơ sở lý thuyết, quá trình thiết kế, triển khai, mô phỏng và đánh giá hệ thống, đồng thời đề xuất các hướng phát triển tiềm năng.

I. CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về IOT

Internet vạn vật (IoT) là một mạng lưới toàn cầu, kết nối hàng tỷ thiết bị vật lý với Internet, cho phép chúng thu thập, chia sẻ và trao đổi dữ liệu với nhau cũng như với các hệ thống trung tâm. Không chỉ giới hạn ở máy tính hay điện thoại, IoT mở rộng phạm vi kết nối đến mọi vật dụng hàng ngày như đồ gia dụng, thiết bị công nghiệp, xe cộ, cảm biến môi trường, v.v., miễn là chúng được tích hợp khả năng cảm biến, xử lý và kết nối mạng.

Một hệ thống IoT điển hình thường được tổ chức theo kiến trúc phân lớp. Mô hình 4 lớp được sử dụng rộng rãi bao gồm: Lớp tri giác (Perceptual Layer) gồm các thiết bị phân cứng và cảm biến thu thập dữ liệu; Lớp mạng (Network Layer) chịu trách nhiệm truyền tải dữ liệu qua các giao thức như Wi-Fi, MQTT; Lớp hỗ trợ (Support Layer) bao gồm các dịch vụ đám mây, cơ sở dữ liệu và xử lý dữ liệu; và cuối cùng là Lớp ứng dụng (Application Layer), nơi cung cấp giao diện và chức năng cho người dùng cuối.

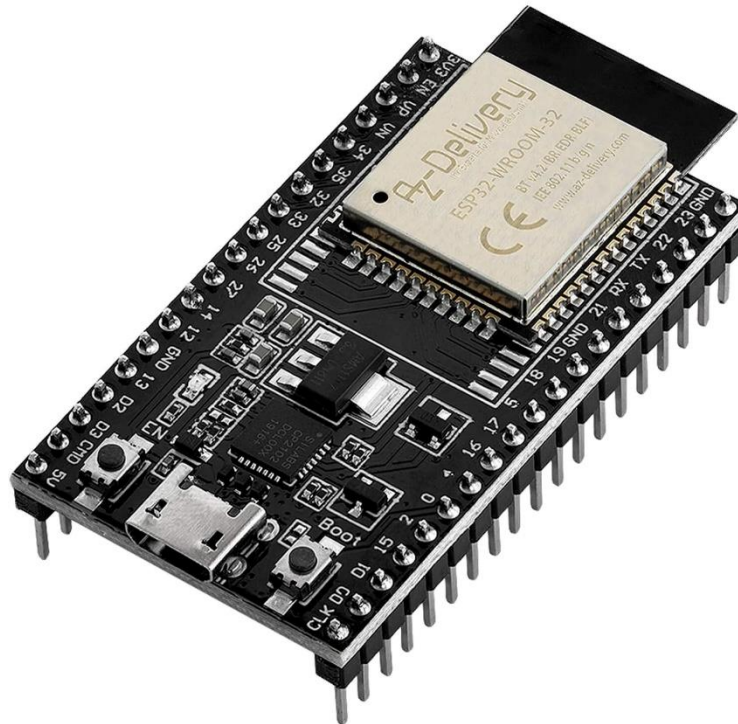
1.2. Vi điều khiển ESP32

ESP32 là một hệ thống trên chip (SoC) vi điều khiển chi phí thấp, tiêu thụ ít năng lượng, được phát triển bởi Espressif Systems. Đây là sự kế thừa và nâng cấp mạnh mẽ của ESP8266, được sử dụng cực kỳ phổ biến trong các dự án IoT. Những đặc điểm nổi bật của ESP32 bao gồm:

- **Tích hợp Wi-Fi và Bluetooth:** ESP32 được tích hợp sẵn cả hai chuẩn kết nối không dây phổ biến là Wi-Fi 802.11 b/g/n (2.4 GHz) và Bluetooth v4.2 BR/EDR và BLE (Bluetooth Low Energy), giúp nó dễ dàng kết nối với Internet và các thiết bị ngoại vi khác.
- **Hiệu năng cao:** Được trang bị vi xử lý lõi kép Tensilica Xtensa LX6 32-bit, có thể hoạt động ở tốc độ lên đến 240 MHz, cùng với bộ nhớ RAM và Flash có dung lượng lớn.

- **Nhiều chân GPIO:** ESP32 cung cấp nhiều chân đầu vào/ra đa năng (GPIO), cho phép giao tiếp với nhiều loại cảm biến, cơ cấu chấp hành, và các thiết bị ngoại vi khác qua các giao thức như I2C, SPI, UART, ADC, DAC.
- **Hỗ trợ lập trình đa dạng:** Có thể lập trình cho ESP32 bằng nhiều ngôn ngữ và framework khác nhau, phổ biến nhất là Arduino IDE (sử dụng ngôn ngữ C/C++) và ESP-IDF (Espressif IoT Development Framework) của chính nhà sản xuất, cùng với MicroPython và các nền tảng khác.

Nhờ những ưu điểm trên, ESP32 là một lựa chọn lý tưởng cho các ứng dụng IoT yêu cầu xử lý tác vụ phức tạp, kết nối không dây ổn định và chi phí hợp lý.



Hình 1: Ảnh thực tế bo mạch ESP32

Nguồn hình ảnh: <https://www.az-delivery.de/fr/products/esp-32-dev-kit-c-v4>

1.3. Giao thức MQTT

MQTT (Message Queuing Telemetry Transport) là một giao thức nhắn tin dạng publish/subscribe, được thiết kế đặc biệt cho các thiết bị IoT có băng thông thấp, độ trễ

cao hoặc hoạt động trong mạng không ổn định. Điểm mạnh của MQTT nằm ở sự đơn giản, nhẹ nhàng và khả năng mở rộng cao.

Mô hình hoạt động của MQTT bao gồm ba thành phần chính:

- **Publisher:** Là các thiết bị hoặc ứng dụng gửi dữ liệu đến một "topic" (chủ đề) cụ thể.
- **Subscriber:** Là các thiết bị hoặc ứng dụng đăng ký nhận dữ liệu từ một hoặc nhiều "topic" mà chúng quan tâm.
- **Broker:** Là một máy chủ trung tâm, có nhiệm vụ tiếp nhận tất cả các bản tin từ Publisher, lọc chúng theo topic và chuyển tiếp đến các Subscriber đã đăng ký nhận topic đó.

Giao thức MQTT cung cấp ba cấp độ chất lượng dịch vụ (QoS - Quality of Service) để đảm bảo việc truyền tin phù hợp với yêu cầu ứng dụng:

- **QoS 0 (At most once):** Bản tin được gửi đúng một lần, không có xác nhận. Đây là cấp độ nhanh nhất nhưng có thể mất tin.
- **QoS 1 (At least once):** Bản tin được gửi ít nhất một lần và đảm bảo đến đích, có thể bị trùng lặp.
- **QoS 2 (Exactly once):** Bản tin được gửi chính xác một lần, đảm bảo không mất mát và không trùng lặp, sử dụng cơ chế bắt tay bốn bước phức tạp hơn.

Ngoài ra, MQTT còn có các tính năng hữu ích như "Retained Messages" (lưu bản tin cuối cùng trên broker) và "Last Will and Testament" (thông báo khi client ngắt kết nối đột ngột).

1.4. TMQTT Broker (Mosquitto/HiveMQ)

MQTT Broker là trung tâm của mọi hệ thống sử dụng MQTT. Nó đóng vai trò như một bưu điện, tiếp nhận, phân loại và chuyển phát tất cả các bản tin giữa các client mà không cần chúng phải biết về sự tồn tại của nhau. Có nhiều loại broker khác nhau, từ các giải pháp mã nguồn mở có thể tự cài đặt đến các dịch vụ đám mây công cộng.

- **Mosquitto:** Đây là một broker MQTT mã nguồn mở, miễn phí và rất phổ biến. Nó nhẹ, dễ cài đặt và cấu hình trên nhiều hệ điều hành, rất thích hợp cho việc phát triển, thử nghiệm hoặc triển khai các hệ thống IoT vừa và nhỏ.
- **HiveMQ:** Là một nền tảng MQTT thương mại mạnh mẽ, cung cấp cả giải pháp tự lưu trữ (HiveMQ CE) và dịch vụ đám mây (HiveMQ Cloud). HiveMQ được thiết kế cho các hệ thống có quy mô lớn, yêu cầu tính sẵn sàng và độ tin cậy cao. Trong dự án này, một broker công cộng miễn phí của HiveMQ (broker.hivemq.com) được sử dụng vì tính tiện lợi và không yêu cầu cấu hình phức tạp.

1.5. Tổng quan về Telegram

Telegram là một ứng dụng nhắn tin đa nền tảng, miễn phí và dựa trên đám mây, nổi tiếng với tốc độ và tính bảo mật. Một trong những tính năng mạnh mẽ nhất của Telegram dành cho nhà phát triển là Telegram Bot API, cho phép tạo ra các "bot" (robot phần mềm) có thể tương tác với người dùng qua tin nhắn.

Bot Telegram hoạt động như một người dùng ảo, có thể nhận lệnh, gửi phản hồi, thông báo, và thậm chí thực hiện các hành động trên các hệ thống bên ngoài thông qua các yêu cầu HTTP tới API của Telegram. Điều này biến Telegram thành một kênh điều khiển và giám sát từ xa cực kỳ tiện lợi cho các thiết bị IoT. Chỉ cần một thiết bị di động có cài đặt Telegram và kết nối Internet, người dùng có thể ra lệnh cho ESP32 bật/tắt đèn hoặc truy vấn trạng thái từ bất kỳ đâu.

1.6. Nền tảng Blynk IoT

Blynk là một nền tảng IoT toàn diện, cung cấp giải pháp end-to-end cho việc kết nối, giám sát và điều khiển thiết bị từ xa. Nó bao gồm ba thành phần chính:

- **Blynk Cloud:** Một máy chủ đám mây mạnh mẽ, đóng vai trò trung gian kết nối thiết bị phần cứng và ứng dụng di động.

- **Blynk Library:** Thư viện cho các vi điều khiển phổ biến (ESP32, Arduino, Raspberry Pi) giúp việc giao tiếp với Blynk Cloud trở nên dễ dàng chỉ với vài dòng code.
- **Blynk App/Console:** Ứng dụng di động (iOS/Android) hoặc giao diện web cho phép người dùng tạo các dashboard trực quan bằng cách kéo-thả các widget (nút nhấn, slider, đồ thị, v.v.) để điều khiển và giám sát thiết bị.

Blynk mang đến trải nghiệm người dùng tuyệt vời với giao diện trực quan, khả năng tùy biến cao và sự ổn định. Tuy nhiên, phiên bản miễn phí có giới hạn về số lượng widget và thiết bị, và nó yêu cầu kết nối Internet liên tục để hoạt động

1.7. So sánh HTTP polling và thư viện native (Blynk lib, UniversalTelegramBot)

Để một vi điều khiển như ESP32 có thể nhận lệnh từ xa, có hai cách tiếp cận phổ biến: chủ động "hỏi" máy chủ liên tục (HTTP polling) hoặc sử dụng các thư viện giao tiếp chuyên dụng (native libraries).

- **HTTP Polling:** Đây là kỹ thuật mà client (ESP32) sẽ gửi các yêu cầu HTTP định kỳ đến máy chủ để kiểm tra xem có lệnh mới hay không.
 - **Ưu điểm:** Cực kỳ đơn giản để cài đặt, không yêu cầu giao thức hay thư viện đặc biệt nào ngoài WiFiClient.
 - **Nhược điểm:** Rất kém hiệu quả. Đa số các yêu cầu sẽ không có dữ liệu mới, gây lãng phí băng thông và năng lượng. Độ trễ cao vì lệnh chỉ được nhận biết sau chu kỳ polling tiếp theo. Không phù hợp với các ứng dụng yêu cầu thời gian thực hoặc hoạt động bằng pin.
- **Thư viện Native (Blynk, UniversalTelegramBot):** Các thư viện này sử dụng các kỹ thuật giao tiếp hai chiều, hiệu quả hơn. **UniversalTelegramBot** sử dụng HTTP long-polling (giữ kết nối chờ dữ liệu) để nhận lệnh gần như ngay lập tức. **Blynk** và **PubSubClient** (cho MQTT) duy trì một kết nối TCP ổn định, cho phép máy chủ chủ động "đẩy" dữ liệu về client khi có sự kiện.
 - **Ưu điểm:** Độ trễ thấp, phản hồi nhanh, tiết kiệm băng thông và năng lượng. Đây là cách tiếp cận chuyên nghiệp cho các ứng dụng IoT thực thụ.

- **Nhược điểm:** Yêu cầu sử dụng các thư viện phức tạp hơn và cần quản lý kết nối.

Trong dự án này, phương pháp sử dụng thư viện native được lựa chọn để đảm bảo tính thời gian thực và hiệu quả của hệ thống.

1.8. Ứng dụng ESP32 với giao thức MQTT

Sự kết hợp giữa ESP32 và MQTT là một giải pháp kinh điển và mạnh mẽ trong thế giới IoT. ESP32, với khả năng kết nối Wi-Fi, đóng vai trò như một MQTT client, có thể dễ dàng kết nối đến một MQTT broker trên Internet. Nó có thể đóng vai trò là một publisher để gửi dữ liệu cảm biến lên đám mây, hoặc là một subscriber để nhận lệnh điều khiển từ người dùng hoặc các hệ thống khác, hoặc cả hai cùng một lúc. Khả năng tự động reconnect khi mất kết nối của các thư viện MQTT giúp hệ thống trở nên ổn định và đáng tin cậy hơn. Ứng dụng của cặp đôi này vô cùng rộng rãi, từ việc điều khiển một bóng đèn đơn giản đến xây dựng các hệ thống giám sát môi trường phức tạp hay tự động hóa trong công nghiệp.

II. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Yêu cầu chức năng

Hệ thống được thiết kế để đáp ứng các yêu cầu chức năng sau:

1. **Kết nối Wi-Fi:** ESP32 phải tự động kết nối đến mạng Wi-Fi được cấu hình sẵn.
2. **Điều khiển thiết bị:** ESP32 có khả năng bật/tắt một thiết bị ngoại vi (đèn LED) thông qua chân GPIO 27.
3. **Điều khiển qua MQTT:** Hệ thống phải subscribe vào topic MQTT `home/led/ThanhNhat` để nhận lệnh "ON" hoặc "OFF" từ các client MQTT khác. Khi nhận lệnh, thực thi và đồng thời publish trạng thái mới lên topic này.

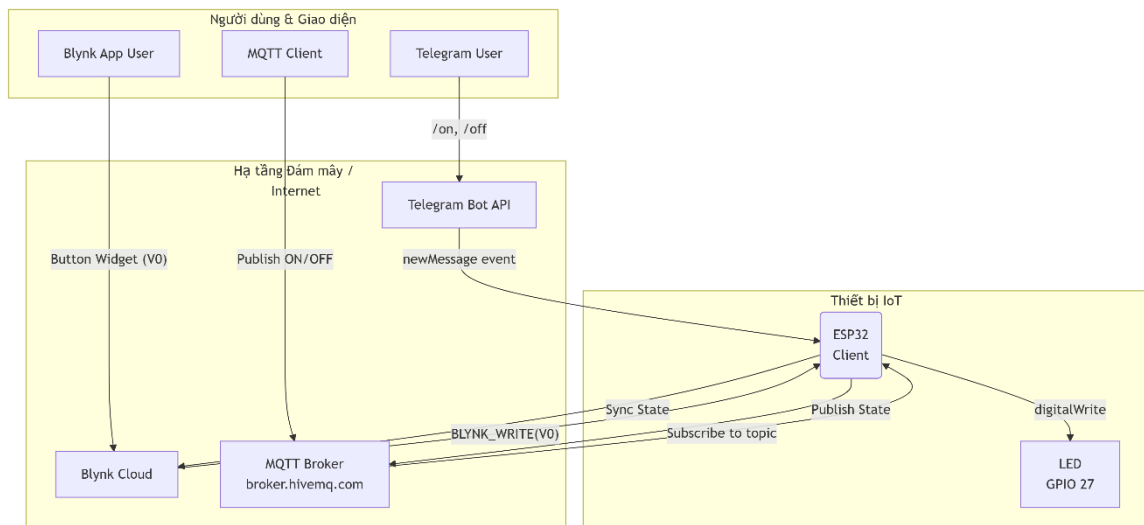
4. **Điều khiển qua Telegram:** Người dùng có thể gửi lệnh /on, /off, /status và /menu từ ứng dụng Telegram để tương tác với bot được cấu hình riêng.
5. **Điều khiển qua Blynk:** Người dùng có thể bật/tắt thiết bị thông qua các widget nút nhấn trên ứng dụng Blynk IoT.
6. **Đồng bộ trạng thái:** Trạng thái của thiết bị phải được đồng bộ trên tất cả các nền tảng. Một thay đổi từ bất kỳ nguồn nào cũng phải được cập nhật lên các nguồn còn lại

2.2. Yêu cầu phi chức năng

1. **Độ tin cậy:** Hệ thống phải có cơ chế tự động kết nối lại khi mất kết nối Wi-Fi hoặc mất kết nối đến MQTT Broker.
2. **Khả năng phản hồi:** Hệ thống phải phản hồi lệnh điều khiển từ các nền tảng một cách nhanh chóng (trong vòng 1-2 giây).
3. **Tính ổn định:** Chương trình phải chạy ổn định, không bị treo, không xảy ra xung đột hoặc lặp lệnh khi có tác động đồng thời từ nhiều nguồn.
4. **Khả năng mở rộng:** Kiến trúc phần mềm phải linh hoạt, dễ dàng mở rộng để điều khiển nhiều thiết bị hơn hoặc tích hợp thêm các nền tảng điều khiển khác.
5. **An toàn (mức độ cơ bản):** Bot Telegram chỉ phản hồi các lệnh từ một `CHAT_ID` được ủy quyền, ngăn chặn sự can thiệp từ bên ngoài.

2.3. Kiến trúc tổng thể hệ thống

Hệ thống được xây dựng dựa trên mô hình client-server với thành phần trung tâm là ESP32 client. Kiến trúc này kết nối ba kênh điều khiển độc lập: MQTT, Telegram và Blynk, tất cả đều hội tụ về thiết bị phân cứng.

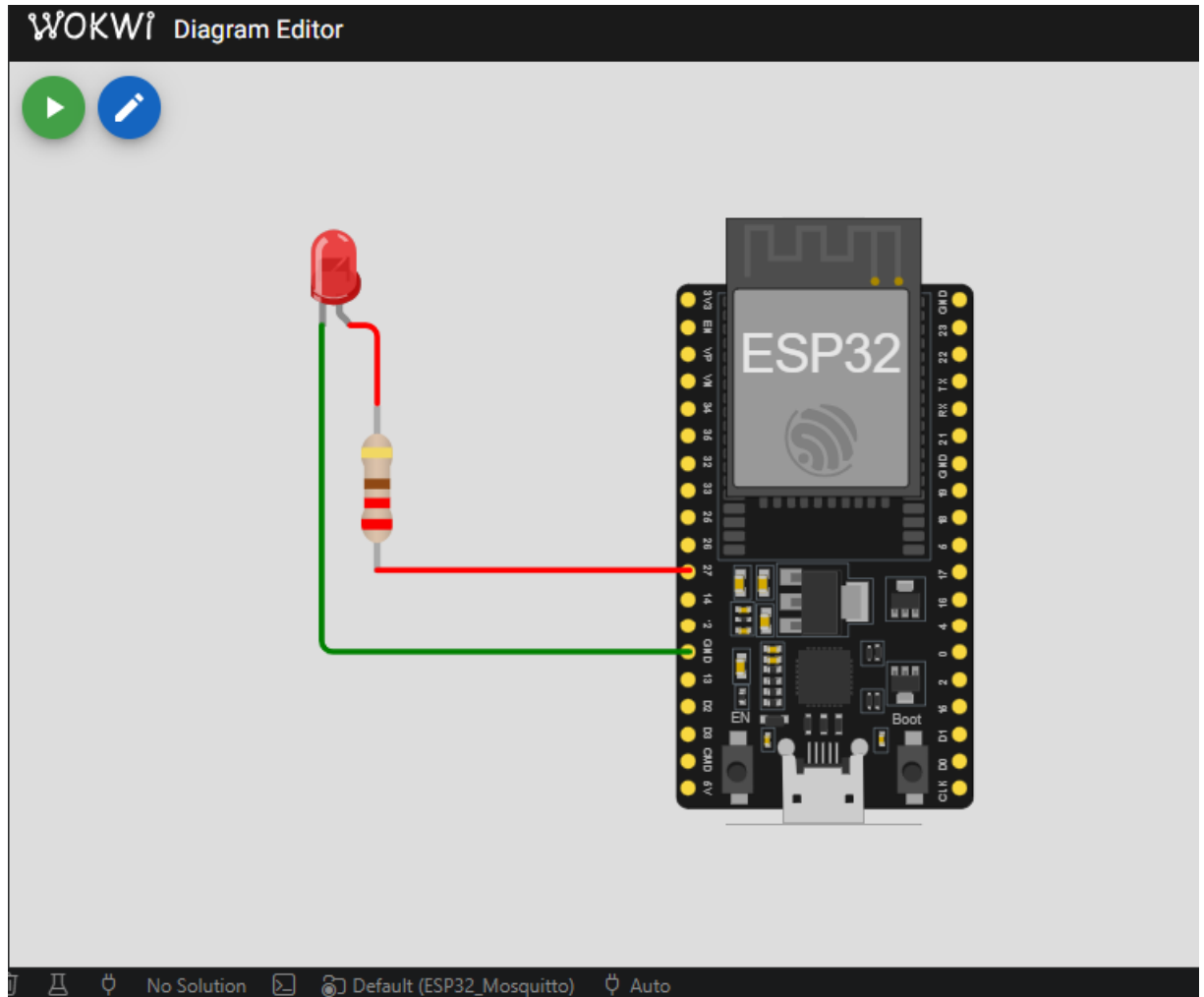


Hình 2: Kiến trúc tổng thể hệ thống

2.4. Thiết kế phần cứng

Sơ đồ phần cứng được thiết kế để tối giản và tập trung vào chức năng điều khiển cơ bản, phù hợp cho mục đích mô phỏng và thử nghiệm.

- **Khối xử lý trung tâm:** Sử dụng board phát triển **ESP32 DevKit-C V4**. Đây là board phổ biến, dễ sử dụng và có đầy đủ các chân GPIO cần thiết.
- **Thiết bị chấp hành:** Một đèn **LED màu đỏ**. Trong thực tế, LED này đại diện cho một thiết bị công suất lớn hơn (ví dụ: đèn chiếu sáng, quạt) sẽ được điều khiển qua một relay.
- **Linh kiện bảo vệ:** Một **điện trở 220Ω**. Điện trở này được mắc nối tiếp với LED để hạn chế dòng điện chạy qua, bảo vệ LED và chân GPIO của ESP32 khỏi bị hỏng.



Hình 3: Sơ đồ mô phỏng của hệ thống

ESP32 (Chân)	Linh kiện	Chân linh kiện
GND.1	LED	C (Cathode)
27 (GPIO27)	Điện trở (R1)	1
-	Điện trở (R1)	2 (nối với LED A)

2.5. Thiết kế phần mềm

Phần mềm cho ESP32 được phát triển trên nền tảng Arduino Framework, tổ chức thành các khối chức năng rõ ràng trong file `main.cpp`.

1. **Khối cấu hình:** Chứa các định nghĩa hằng số như `LED_PIN`, thông tin Wi-Fi (`WIFI_SSID`, `WIFI_PASS`), cấu hình MQTT (`MQTT_HOST`, `TOPIC`), Token Blynk và Token Telegram Bot.
2. **Khối điều khiển thiết bị (Core Logic):**
 - `writeLedRaw(bool on)`: Hàm mức thấp ghi tín hiệu `HIGH/LOW` trực tiếp lên chân GPIO.
 - `setLed(bool st, bool syncBlynk = true)`: Hàm trung tâm để thay đổi trạng thái LED. Nó gọi `writeLedRaw` và có nhiệm vụ đồng bộ trạng thái mới lên Blynk Cloud.
 - `applyFromSource(bool st, const char *source)`: Hàm "thông minh" được gọi mỗi khi có lệnh từ bất kỳ nguồn nào (Telegram, Blynk, MQTT). Nó gọi `setLed`, cập nhật thông tin về nguồn gốc lệnh cuối cùng để chống lặp, và quyết định có cần publish trạng thái lên MQTT hay không (nếu lệnh không đến từ MQTT).
3. **Khối kết nối MQTT:**
 - Sử dụng thư viện `PubSubClient`.
 - Hàm `mqttCallback` xử lý các bản tin nhận được từ topic đã subscribe.
 - Hàm `ensureMqtt` đảm bảo kết nối MQTT luôn được duy trì và tự động reconnect khi cần.
4. **Khối giao tiếp Telegram:**
 - Sử dụng thư viện `UniversalTelegramBot`.
 - Hàm `telegramLoop` được gọi định kỳ để kiểm tra tin nhắn mới.
 - Hàm `handleTelegramMessages` phân tích cú pháp lệnh và gọi `applyFromSource` để thực thi.
5. **Khối giao tiếp Blynk:**
 - Sử dụng thư viện `BlynkSimpleEsp32`.
 - Hàm `BLYNK_WRITE(V0)` là hàm callback được Blynk tự động gọi khi có sự kiện từ widget trên ứng dụng di động.
 - Cấu hình VirtualPin

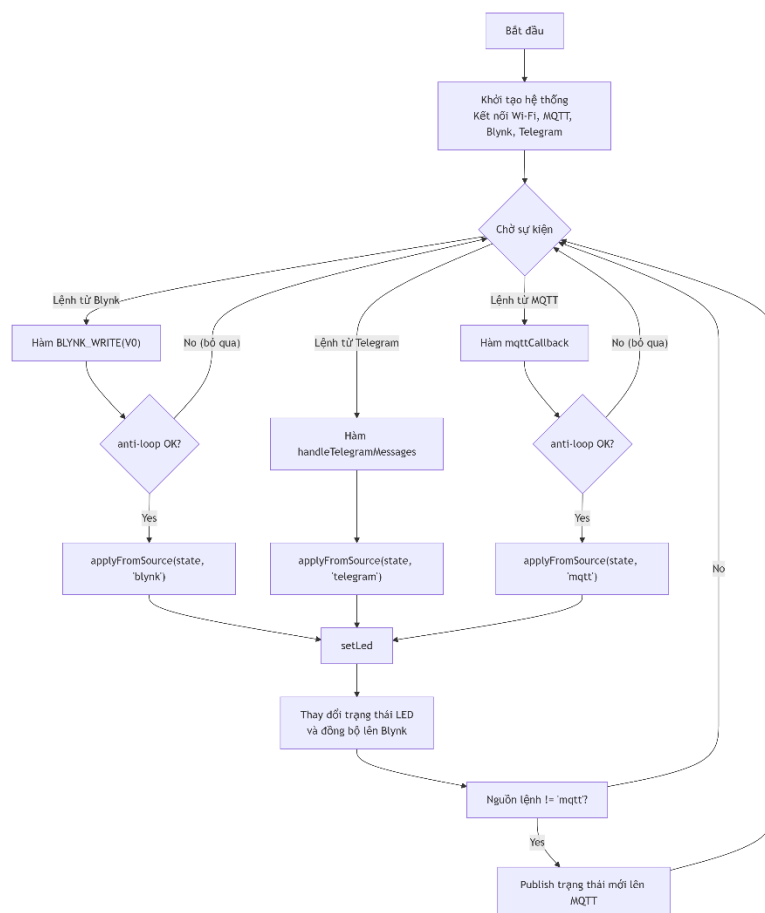
Virtual Pin	Widget	Chức năng
V0	BUTTON	Nút Tắt/Bật điều khiển hệ thống
V1	LED	Hiển thị trạng thái đèn

6. Chương trình chính (main loop):

- Vòng lặp `loop()` liên tục duy trì kết nối Wi-Fi và Blynk, gọi `Blynk.run()`, `ensureMqtt()`, `mqttClient.loop()`, và `telegramLoop()`.

2.6. Lưu đồ thuật toán điều khiển

Lưu đồ thuật toán dưới đây mô tả cách ESP32 xử lý lệnh từ nhiều nguồn, với trọng tâm là cơ chế `applyFromSource` để quản lý và đồng bộ trạng thái.



Hình 4: Lưu đồ thuật toán điều khiển

2.7. Cơ chế đồng bộ trạng thái đa nền tảng (MQTT – Telegram - Blynk)

Sự tồn tại của ba nền tảng điều khiển độc lập tạo ra một thách thức về tính nhất quán của trạng thái hệ thống. Giải pháp là triển khai một cơ chế đồng bộ dựa trên sự kiện, với ESP32 là trung tâm xử lý.

- **Khi có lệnh từ Blynk (applyFromSource với source="blynk"):**

1. Hàm setLed được gọi, thay đổi trạng thái vật lý của LED và cập nhật giá trị của widget V0 trên Blynk (để phản ánh đúng trạng thái).
2. Do source không phải là "mqtt", chương trình sẽ tự động publish trạng thái mới (ON/OFF) lên topic MQTT home/led/ThanhNhat. Điều này giúp mọi MQTT client khác đều biết trạng thái hiện tại của thiết bị.
3. Telegram không được cập nhật chủ động vì nó là giao diện "kéo" (người dùng chủ động hỏi bằng lệnh /status).

- **Khi có lệnh từ Telegram (applyFromSource với source="telegram"):** Quy trình tương tự như trên. Trạng thái được cập nhật lên phần cứng, đồng bộ với Blynk và publish lên MQTT.

- **Khi có lệnh từ MQTT (applyFromSource với source="mqtt"):**

1. setLed thay đổi trạng thái phần cứng và đồng bộ lên Blynk.
2. Do source là "mqtt", chương trình sẽ **KHÔNG** publish lại trạng thái lên MQTT. Điều này ngăn chặn một vòng lặp vô hạn (echo loop), nơi ESP32 vừa nhận lệnh, thực thi, rồi lại publish ra đúng lệnh nó vừa nhận, khiến nó lại nhận lại lệnh đó.

2.8. Cơ chế chống lặp lệnh (anti-loop), chống ghi đè trạng thái

Để đảm bảo hệ thống hoạt động ổn định và không bị rối loạn bởi các tín hiệu điều khiển chồng chéo hoặc đối ngược, các cơ chế sau đã được triển khai:

1. **Chặn xung sync ban đầu của Blynk:** Khi mới kết nối thành công đến Blynk Cloud, Blynk sẽ tự động đồng bộ trạng thái của tất cả các widget. Điều này có thể gây ra một lệnh "ảo" (ví dụ: gửi lệnh tắt đèn) ngay khi

ESP32 vừa khởi động, trong khi trạng thái mong muốn có thể là bật. Để khắc phục, một biến `blynkReady` được sử dụng để chặn tất cả các callback từ Blynk trong 2 giây đầu tiên sau khi kết nối (`blynkReady = false`). Sau 2 giây, `blynkReady` được đặt thành `true` và Blynk mới chính thức lắng nghe lệnh từ người dùng.

2. **Chặn echo từ MQTT và Blynk (Anti-loop logic):** Đây là cơ chế quan trọng nhất. Mỗi khi có một lệnh được thực thi, hai biến toàn cục là `lastCommandMs` (thời gian) và `lastCommandSource` (nguồn gốc) được cập nhật. Trong các hàm callback (`mqttCallback` và `BLYNK_WRITE`), một bước kiểm tra được thực hiện: nếu lệnh mới đến trong vòng 1500ms (1.5 giây) kể từ lệnh cuối cùng và `lastCommandSource` không phải là chính nguồn đó, lệnh mới sẽ bị bỏ qua. Điều này ngăn chặn hiệu quả các vòng lặp vô hạn và sự ghi đè lẫn nhau giữa các kênh điều khiển.

III. Triển khai thực nghiệm

3.1. Công cụ và môi trường (Wokwi, PlatformIO, Arduino framework)

Việc triển khai dự án sử dụng các công cụ và môi trường phát triển hiện đại, giúp đẩy nhanh quá trình phát triển và giảm thiểu rủi ro phần cứng.

- **Wokwi:** Là một nền tảng mô phỏng phần cứng trực tuyến mạnh mẽ, cho phép mô phỏng các vi điều khiển như ESP32, Arduino cùng với các linh kiện ngoại vi. Wokwi được sử dụng để xây dựng sơ đồ nguyên lý, viết code và chạy thử nghiệm toàn bộ hệ thống trong môi trường ảo trước khi triển khai trên phần cứng thật, tiết kiệm thời gian và chi phí.
- **Arduino Framework:** Đây là framework lập trình được sử dụng do tính đơn giản, dễ tiếp cận và có một cộng đồng hỗ trợ rộng lớn. Code được viết bằng ngôn ngữ C/C++ với các hàm API quen thuộc của Arduino (`pinMode`, `digitalWrite`, `delay`, v.v.).
- **PlatformIO (Tùy chọn):** Dù dự án có thể biên dịch trực tiếp trên Wokwi, việc phát triển code có thể được thực hiện trên PlatformIO IDE (một extension cho

VSCode) để tận dụng các tính năng quản lý thư viện chuyên nghiệp, hỗ trợ debug và biên dịch nhanh hơn.

3.2. Cấu hình Wifi, MQTT, Telegram, Blynk

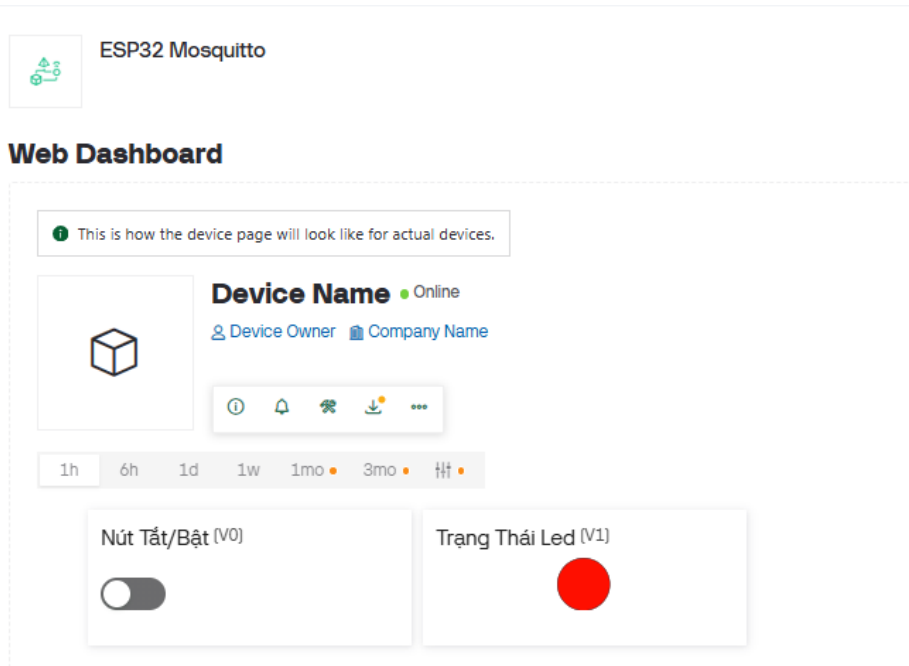
Các bước cấu hình cần thiết trước khi nạp code:

1. **Wi-Fi:** Cập nhật `WIFI_SSID` và `WIFI_PASS` trong code để phù hợp với mạng Wi-Fi mà ESP32 sẽ kết nối. Trong môi trường Wokwi, SSID "Wokwi-GUEST" là một mạng ảo mặc định.
2. **MQTT:** Broker công cộng `broker.hivemq.com` và cổng `1883` được sử dụng. Topic `home/led/ThanhNhat` có thể được thay đổi tùy ý.
3. **Blynk:** Người dùng cần tạo một "Template" và một "Device" trên Blynk Cloud.

Các ID và Token quan trọng bao

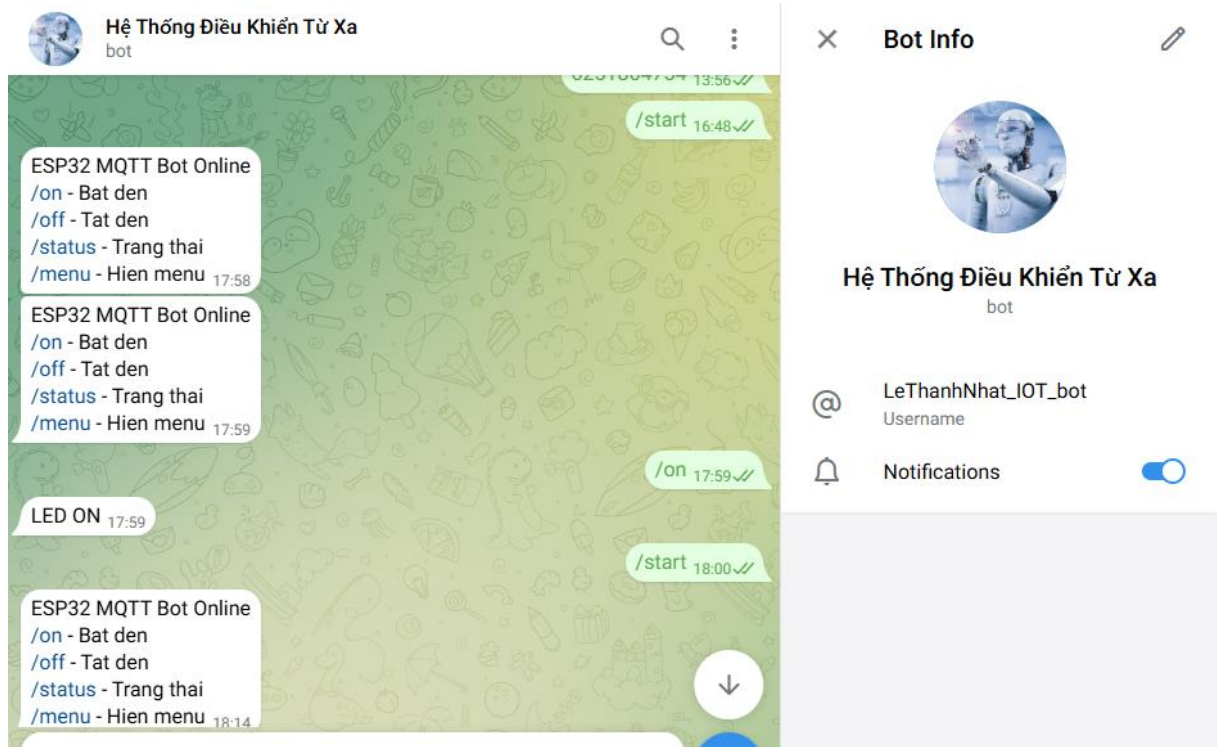
gồm `BLYNK_TEMPLATE_ID`, `BLYNK_TEMPLATE_NAME`,

và `BLYNK_AUTH_TOKEN` sẽ được sao chép vào code. Trên Web Dashboard hoặc Blynk App, cần tạo hai widget: một Button (gán cho chân ảo V0) và một LED/Switch (gán cho chân ảo V1) để hiển thị trạng thái.



Hình 5: Cấu hình Dashboard Blynk

4. **Telegram:** Sử dụng BotFather trên Telegram để tạo một bot mới. Sau khi tạo, BotFather sẽ cung cấp một `BOT_TOKEN`. Người dùng cũng cần lấy `CHAT_ID` của chính mình (bằng cách gửi tin nhắn cho bot và truy cập API `getUpdates`).



Hình 4: Giao diện Telegram sau khi đã cấu hình

3.3. Cài đặt thư viện và biên dịch

Để biên dịch thành công, cần cài đặt các thư viện sau trong Arduino IDE hoặc khai báo trong file `platformio.ini`:

- **PubSubClient (by Nick O'Leary):** Thư viện cho giao thức MQTT.
- **Blynk (by Volodymyr Shymanskyy):** Thư viện kết nối Blynk.
- **UniversalTelegramBot (by Brian Lough):** Thư viện giao tiếp với Telegram Bot API.
- **ArduinoJson (by Benoit Blanchon):** Thư viện xử lý JSON, cần thiết cho `UniversalTelegramBot`.

3.4. Nạp chương trình và vận hành

1. Trong môi trường Wokwi, chỉ cần nhấn nút "Start Simulation". Trình mô phỏng sẽ tự động biên dịch code và chạy chương trình trên ESP32 ảo. Serial Monitor sẽ hiển thị quá trình khởi động và các thông báo gỡ lỗi.
2. Vận hành:
 - Sau khi khởi động, LED sẽ sáng trong 3 giây để báo hiệu hệ thống đang hoạt động, sau đó tắt.
 - ESP32 sẽ kết nối Wi-Fi, sau đó kết nối đến MQTT Broker, Blynk Cloud, và Telegram Bot.
 - Người dùng có thể gửi lệnh /on hoặc /off từ Telegram để điều khiển LED.
 - Sử dụng nút bấm trên ứng dụng Blynk để bật/tắt LED.
 - Sử dụng bất kỳ MQTT client nào (ví dụ: MQTTS, MQTT.fx) để publish "ON" hoặc "OFF" vào topic home/led/ThanhNhat.
 - Quan sát trạng thái LED thay đổi trên cả giao diện Wokwi lẫn các ứng dụng điều khiển.

IV. MÔ PHỎNG TRÊN WOKWI VÀ ĐÁNH GIÁ HỆ THỐNG

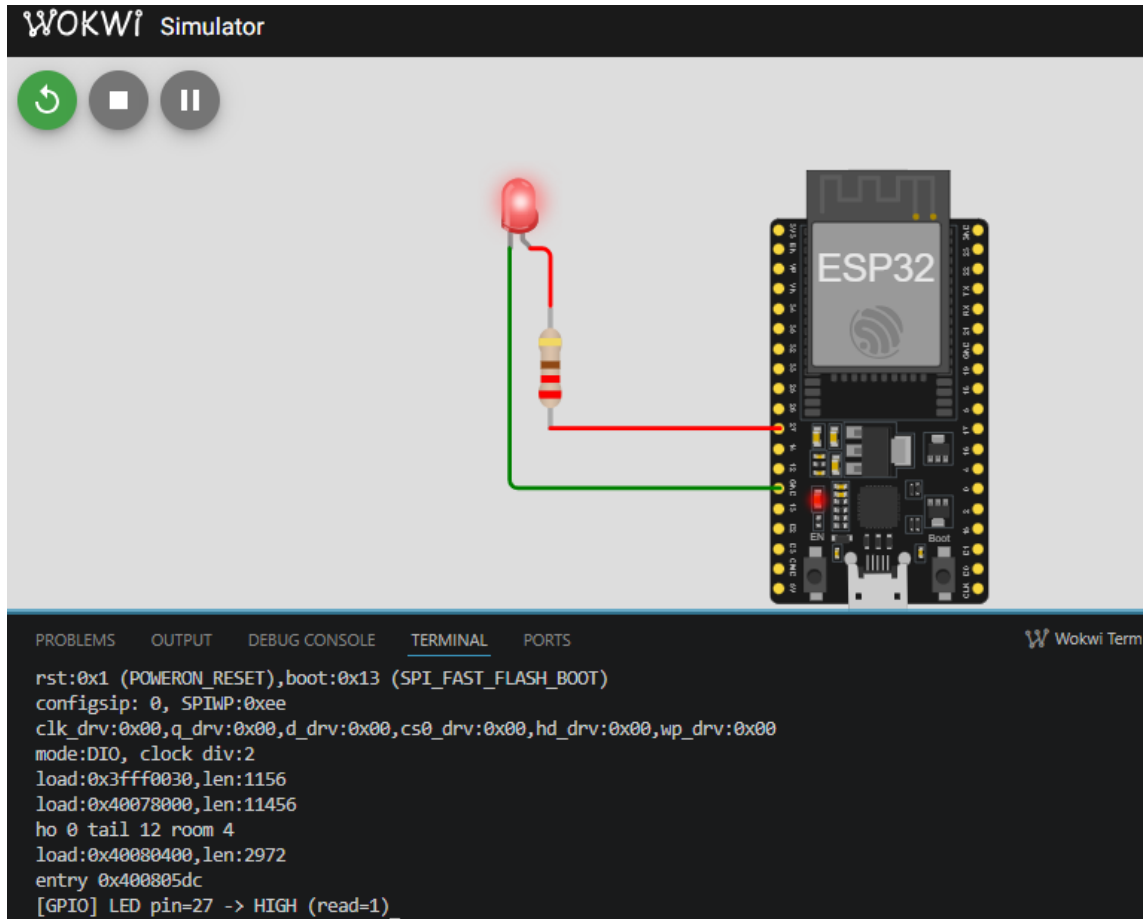
4.1. Giới thiệu Wokwi

Wokwi là một công cụ mô phỏng điện tử trực tuyến, cho phép người dùng tạo và chia sẻ các dự án phần cứng mà không cần sở hữu linh kiện thật. Nó hỗ trợ mô phỏng một loạt các vi điều khiển phổ biến (ESP32, Arduino, STM32, Pi Pico) và một thư viện lớn các linh kiện như LED, điện trở, cảm biến, màn hình, và thậm chí cả các module Wi-Fi. Wokwi không chỉ là một công cụ vẽ sơ đồ mạch; nó thực sự chạy firmware đã biên dịch trong một máy ảo, cho phép debug từng bước, xem Serial Monitor, và tương tác với các linh kiện ảo giống như trên thực tế. Đối với các dự án IoT, Wokwi còn hỗ trợ mô phỏng kết nối Wi-Fi, giúp ESP32 ảo có thể kết nối thực sự đến Internet và giao tiếp với các dịch vụ đám mây như MQTT Broker hay Blynk.

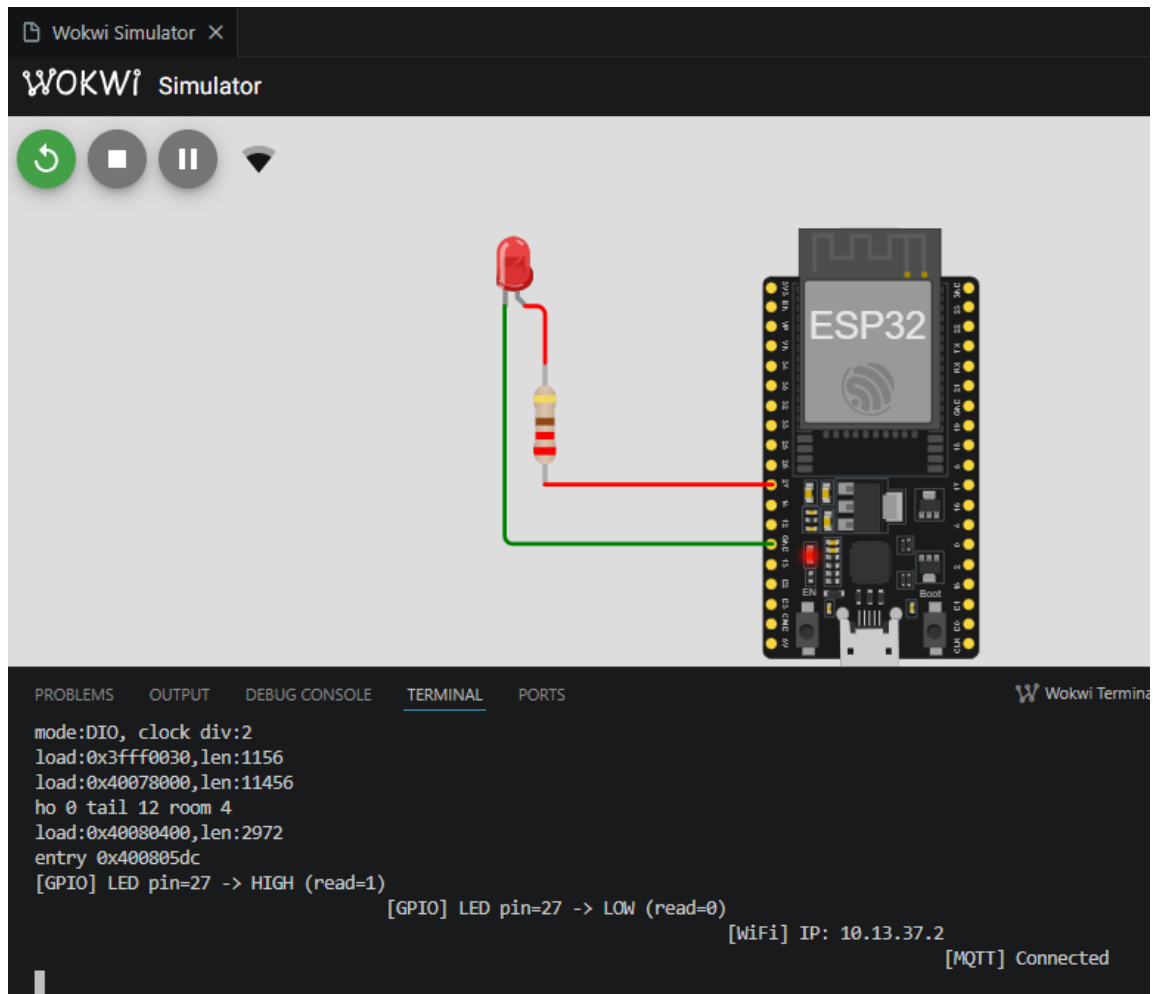
4.2. Kết quả mô phỏng

Quá trình mô phỏng trên Wokwi đã chứng minh tính đúng đắn của hệ thống:

1. **Khởi động và kết nối:** Khi nhấn "Start Simulation", Serial Monitor hiển thị quá trình ESP32 kết nối thành công đến Wi-Fi, MQTT Broker và Blynk Cloud. LED trên sơ đồ nhấp nháy 3 giây như thiết kế.

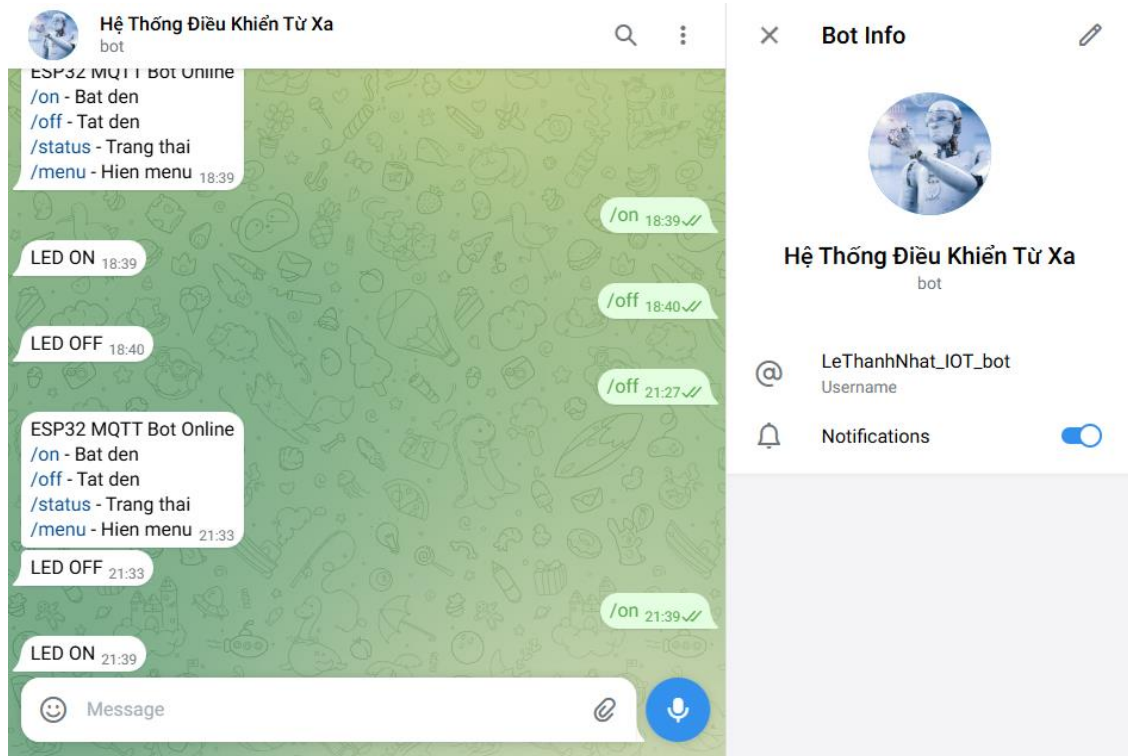


Hình 6: Khi khởi động đèn nhấp nháy



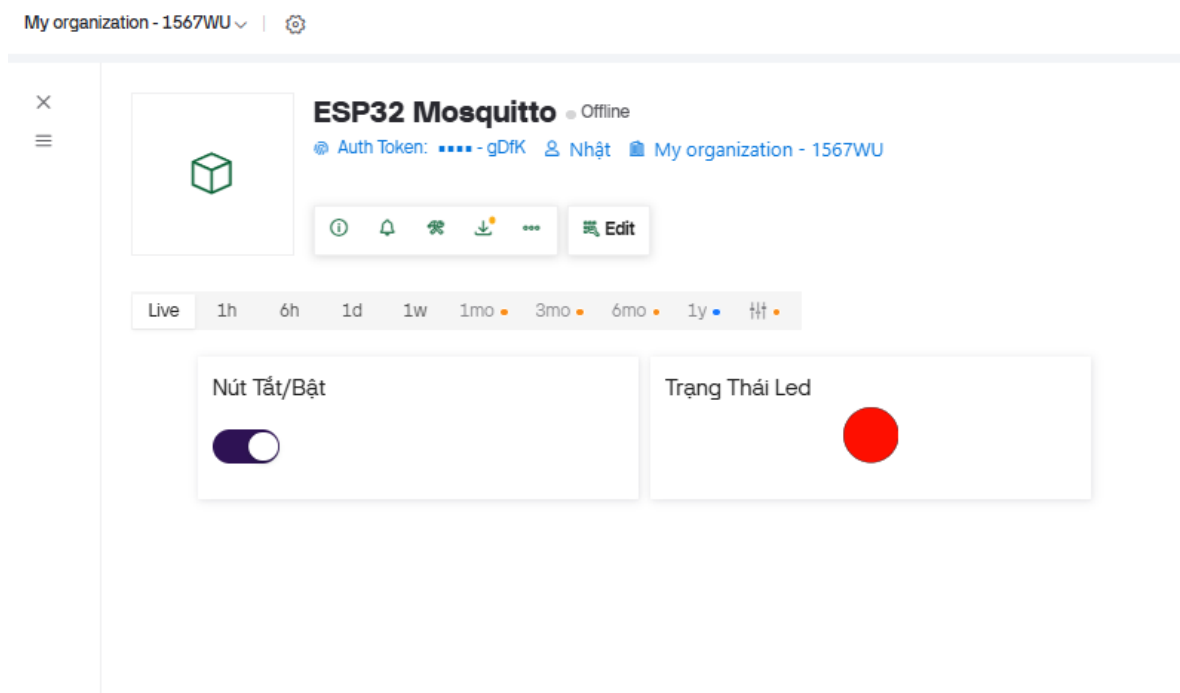
Hình 7: Kết nối tới wifi, MQTT, Blynk, Telegram

2. **Điều khiển qua Telegram:** Gửi tin nhắn /on từ Telegram, LED trên Wokwi lập tức sáng. Serial Monitor ghi nhận "[LED] ON by telegram". Gửi /off, LED tắt, Serial ghi "[LED] OFF by telegram". Lệnh /status trả về đúng trạng thái của LED.



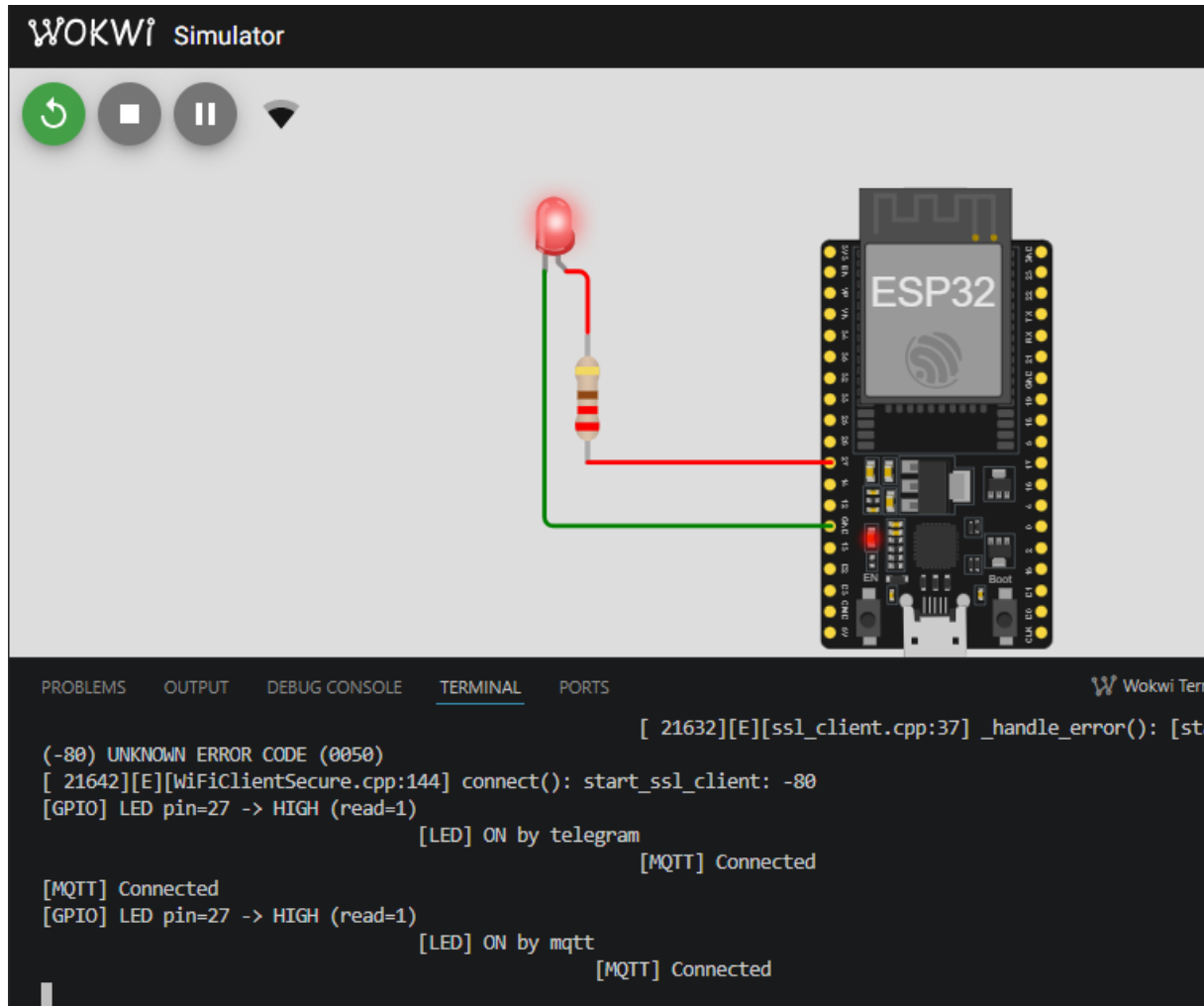
Hình 8: Điều khiển qua Telegram

3. **Điều khiển qua Blynk:** Bật/tắt nút nhấn V0 trên Blynk Web Dashboard, LED trên Wokwi phản hồi tức thì. Trạng thái của widget V1 cũng thay đổi tương ứng. Serial ghi "[LED] ON by blynk" hoặc "[LED] OFF by blynk".



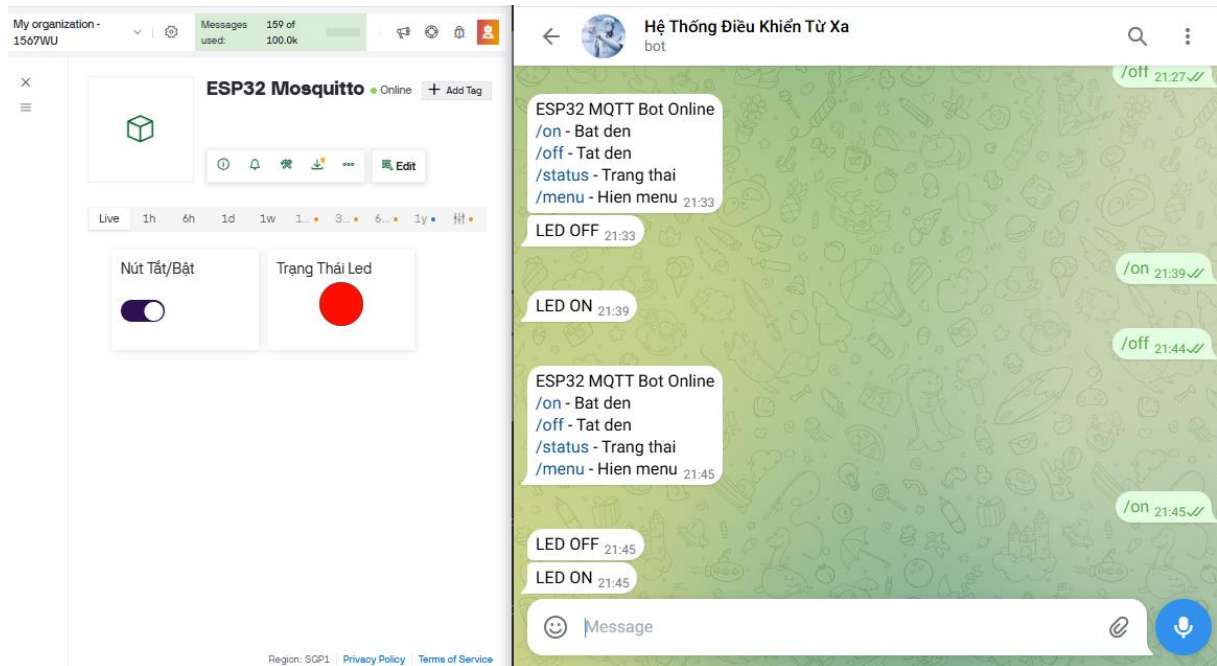
Hình 9: Điều khiển tắt/bật qua blynk và hiển thị trạng thái LED

4. **Điều khiển qua MQTT:** Sử dụng một MQTT client khác publish "ON" vào topic `home/led/ThanhNhat`. LED sáng. Serial ghi "[LED] ON by mqtt"



Hình 10: Điều khiển qua MQTT

5. **Đồng bộ trạng thái:** Mọi thay đổi từ một nguồn đều được phản ánh chính xác trên các nguồn còn lại. Ví dụ, khi điều khiển LED bằng Telegram, nút nhấn trên Blynk tự động chuyển sang trạng thái "ON".



Hình 11: Đồng bộ trạng thái

6. **Chống lặp lệnh:** Serial Monitor cho thấy các thông báo "[MQTT] ignore echo/override" hoặc "[BLYNK] ignore override" khi cơ chế anti-loop được kích hoạt, chứng tỏ hệ thống đã xử lý đúng các tình huống xung đột tiềm ẩn.

4.3. Đánh giá hệ thống và đề xuất hướng phát triển

4.3.1. Đánh giá:

Ưu điểm:

- Hệ thống hoạt động ổn định, đáp ứng đầy đủ các yêu cầu chức năng đề ra.
- Kiến trúc phần mềm rõ ràng, module hóa tốt, dễ dàng bảo trì và mở rộng.
- Cơ chế chống lặp và chặn xung sync ban đầu của Blynk hoạt động hiệu quả, đảm bảo tính nhất quán dữ liệu.
- Việc sử dụng Wokwi giúp quá trình phát triển và kiểm thử diễn ra nhanh chóng và an toàn.

Hạn chế:

- Hệ thống hiện chỉ điều khiển một thiết bị duy nhất.

- Thiếu cơ chế lưu trữ dữ liệu lịch sử (ví dụ: nhật ký bật/tắt).
- Giao diện điều khiển còn đơn giản, chưa có các tính năng nâng cao như hẹn giờ, điều khiển theo lịch trình.
- Việc cập nhật firmware vẫn cần thực hiện thủ công qua cáp USB.

4.3.2 Hướng phát triển:

1. **Đa thiết bị:** Mở rộng hệ thống để điều khiển nhiều thiết bị cùng lúc (ví dụ: nhiều đèn, quạt, ổ cắm) bằng cách sử dụng nhiều chân GPIO và cấu trúc dữ liệu mảng hoặc JSON để quản lý trạng thái.
2. **Dashboard nâng cao:** Xây dựng một giao diện web (web dashboard) riêng trên chính ESP32 hoặc trên một máy chủ khác, cung cấp khả năng giám sát và điều khiển trực quan hơn, không phụ thuộc vào Blynk.
3. **Cơ sở dữ liệu:** Tích hợp cơ sở dữ liệu (ví dụ: MySQL, InfluxDB) trên một máy chủ từ xa để lưu trữ lịch sử hoạt động của thiết bị, từ đó phân tích và đưa ra các thống kê.
4. **OTA (Over-The-Air) firmware:** Triển khai tính năng cập nhật firmware từ xa qua Wi-Fi. Điều này cho phép nâng cấp, sửa lỗi hoặc bổ sung tính năng mới cho ESP32 mà không cần phải tháo thiết bị để nạp code bằng dây cáp, một yếu tố then chốt cho các sản phẩm IoT thương mại.

V. KẾT LUẬN

Đề tài "Điều khiển từ xa qua giao thức MQTT với ESP32" đã được nghiên cứu và triển khai thành công dưới dạng một hệ thống mô phỏng hoàn chỉnh. Bài tiểu luận đã trình bày một cách có hệ thống từ cơ sở lý thuyết về IoT, ESP32 và MQTT đến quá trình phân tích, thiết kế và hiện thực hóa một giải pháp điều khiển đa nền tảng, tích hợp cùng lúc ba kênh tương tác phổ biến là MQTT, Telegram và Blynk. Kết quả mô phỏng trên nền tảng Wokwi đã chứng minh tính khả thi và ổn định của hệ thống, đặc biệt là hiệu quả của các cơ chế đồng bộ trạng thái và chống lặp lệnh.

Dự án này không chỉ cung cấp một mô hình thực tế cho việc xây dựng các ứng dụng IoT điều khiển từ xa mà còn mở ra nhiều hướng phát triển tiềm năng, hướng tới một hệ thống thông minh và chuyên nghiệp hơn. Những kiến thức và kỹ năng thu được từ quá trình thực hiện là nền tảng vững chắc để tiếp tục nghiên cứu và phát triển các giải pháp IoT phức tạp hơn trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] ZenoPCB, “ESP32 Gửi Dữ Liệu MQTT Qua Wi-Fi và Điều Khiển LED.” [Online]. Available: <https://zenopcb.vn/esp32-gui-du-lieu-mqtt-qua-wi-fi-va-dieu-khien-led/>
- [2] Amazon Web Services, “Internet vạn vật (IoT) là gì?” [Online]. Available: <https://aws.amazon.com/vi/what-is/iot/>
- [3] TAPIT, “Internet of Things: Định nghĩa và kiến trúc.” [Online]. Available: <https://tapit.vn/internet-things-dinh-nghia-va-kien-truc/>
- [4] IoTZone, “Cách dùng ESP32 Telegram điều khiển đèn LED với Arduino IDE.” [Online]. Available: <https://iotzone.vn/esp32/cach-dung-esp32-telegram-dieu-khien-den-led-voi-arduino-ide/>
- [5] Điện Thông Minh E-smart, “Cách dùng Blynk IOT với ESP32 điều khiển giám sát thiết bị.” [Online]. Available: <https://dienthongminhesmart.com/lap-trinh-esp32/blynk-iot-va-esp32/>
- [6] iMIC, “Tìm hiểu OTA – Giải pháp cập nhật từ xa giúp IoT vận hành bền vững.” [Online]. Available: <https://imic.edu.vn/tim-hieu-ota-giai-phap-cap-nhat-tu-xa-giup-iot-van-hanh-ben-vung>
- [7] Espressif Systems, “ESP32 Series Datasheet.” [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [8] [MQTT.org](https://mqtt.org/), “MQTT - The Standard for IoT Messaging.” [Online]. Available: <https://mqtt.org/>
- [9] HiveMQ, “HiveMQ Community Edition.” [Online]. Available: <https://www.hivemq.com/>
- [10] Telegram, “Telegram Bot API.” [Online]. Available: <https://core.telegram.org/bots/api>
- [11] Blynk Inc., “Blynk IoT Platform Documentation.” [Online]. Available: <https://docs.blynk.io/>
- [12] Wokwi, “Wokwi - Online ESP32, STM32, Arduino Simulator.” [Online]. Available: <https://wokwi.com/>
- [13] Espressif Systems, “Over-The-Air (OTA) Updates.” [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/ota.html>

[14] Espressif Systems, “ESP32 Arduino Core Documentation.” [Online].

Available: <https://espressif-docs.readthedocs-hosted.com/projects/arduino-esp32/en/latest/>

[15] Random Nerd Tutorials, “Telegram: Control ESP32/ESP8266 Outputs with Arduino

IDE.” [Online]. Available: <https://randomnerdtutorials.com/telegram-control-esp32-esp8266-nodemcu-outputs/>

CBCht2
(Ký và ghi rõ họ tên)